W3Schools Tutorials Exercises
W3Schools app NEW iOS & Android
Sign In
. CSS JAVASCRIPT SQL PYTHON JAVA PHP C C++ C# AWS

React Forms

‹ Previous Next ›

Just like in HTML, React uses forms to allow users to interact with the web page.

Adding Forms in React

SHARE

You add a form with React like any other element:

Example:

Get your own React.js Server

Add a form that allows users to enter their name:

```
function MyForm() {
  return (
    <form>
      <label>Enter your name:
        <input type="text" />
      </label>
```

W3Schools Tutorials Exercises W3Schools app iOS & Android NEW Sign In
. CSS JAVASCRIPT SQL PYTHON JAVA PHP C C++ C# AWS

```
createRoot(document.getElementById('root')).render(
  <MyForm />
);
```

Run Example »

This will work as normal, the form will submit and the page will refresh.

But this is generally not what we want to happen in React.

We want to prevent this default behavior and let React control the form.

HTML Forms vs. React Forms

In React, form elements like `<input>`, `<textarea>`, and `<select>` work a bit differently from traditional HTML.

In standard HTML, form elements maintain their own value based on user input.

For example, an `<input type="text">` field keeps track of its own value in the HTML DOM.

In React, the value of the form element is kept in the component's `state` property and updated *only* with the `setState()` function.

In other words; React provides a way to manage form data through component state, leading to what are known as "controlled components."

Tutorials Exercises W3Schools app iOS & Android NEW Sign In
CSS JAVASCRIPT SQL PYTHON JAVA PHP C C++ C# AWS

REMOVE ADS

Controlled Components

In a controlled component, form data is handled by the React component.

The value of the input element is driven by the React *state*, and any changes to that value are managed through event handlers that update the state.

When the data is handled by the components, all the data is stored in the component state.

We can use the `useState` Hook to keep track of each input value and provide a "single source of truth" for the entire application.

See the React Hooks section for more information on Hooks.

Example:

Use the `useState` Hook to manage the input:

```
import { useState } from 'react';
import { createRoot } from 'react-dom/client';

function MyForm() {
  const [name, setName] = useState("");

  function handleChange(e) {
    setName(e.target.value);
  }
```

Tutorials Exercises W3Schools app iOS & Android NEW Sign In
. CSS JAVASCRIPT SQL PYTHON JAVA PHP C C++ C# AWS

```
        <input
          type="text"
          value={name}
          onChange={handleChange}
        />
      </label>
      <p>Current value: {name}</p>
    </form>
  )
}

createRoot(document.getElementById('root')).render(
  <MyForm />
);
```

Run Example »

Example Explained:

1. Import the `useState` Hook from React:

```
import { useState } from 'react';
```

2. Declare a state variable to hold the input's value and a function to update it:

```
const [name, setName] = useState("");
```

3. Create a function to handle the change event:

```
function handleChange(e) {
  setName(e.target.value);
```

W3Schools Tutorials Exercises W3Schools app iOS & Android NEW Sign In
. CSS JAVASCRIPT SQL PYTHON JAVA PHP C C++ C# AWS

change event:

```
<input
  type="text"
  value={name}
  onChange={handleChange}
/>
```

5. Display the current value to show that the value is being updated:

```
<p>Current value: {name}</p>
```

Initial Values

To add an initial value to the input field in the example above, add a value to the `useState` object:

Example:

Use initial value for name:

```
function MyForm() {
  const [name, setName] = useState("John");

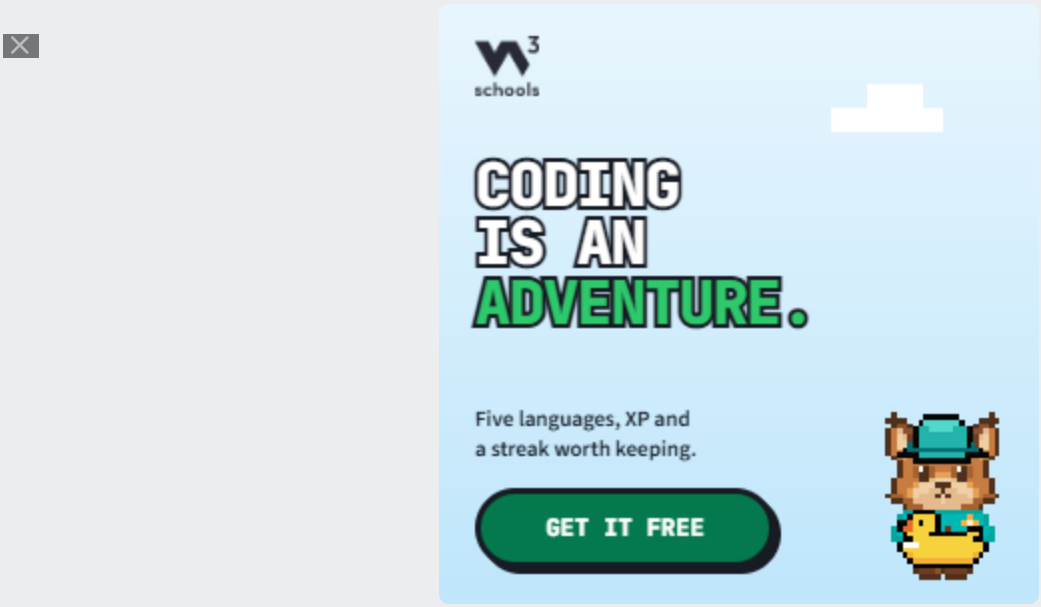
  ...
```

Run Example »

Tutorials Exercises W3Schools app iOS & Android NEW Sign In

. CSS JAVASCRIPT SQL PYTHON JAVA PHP C C++ C# AWS

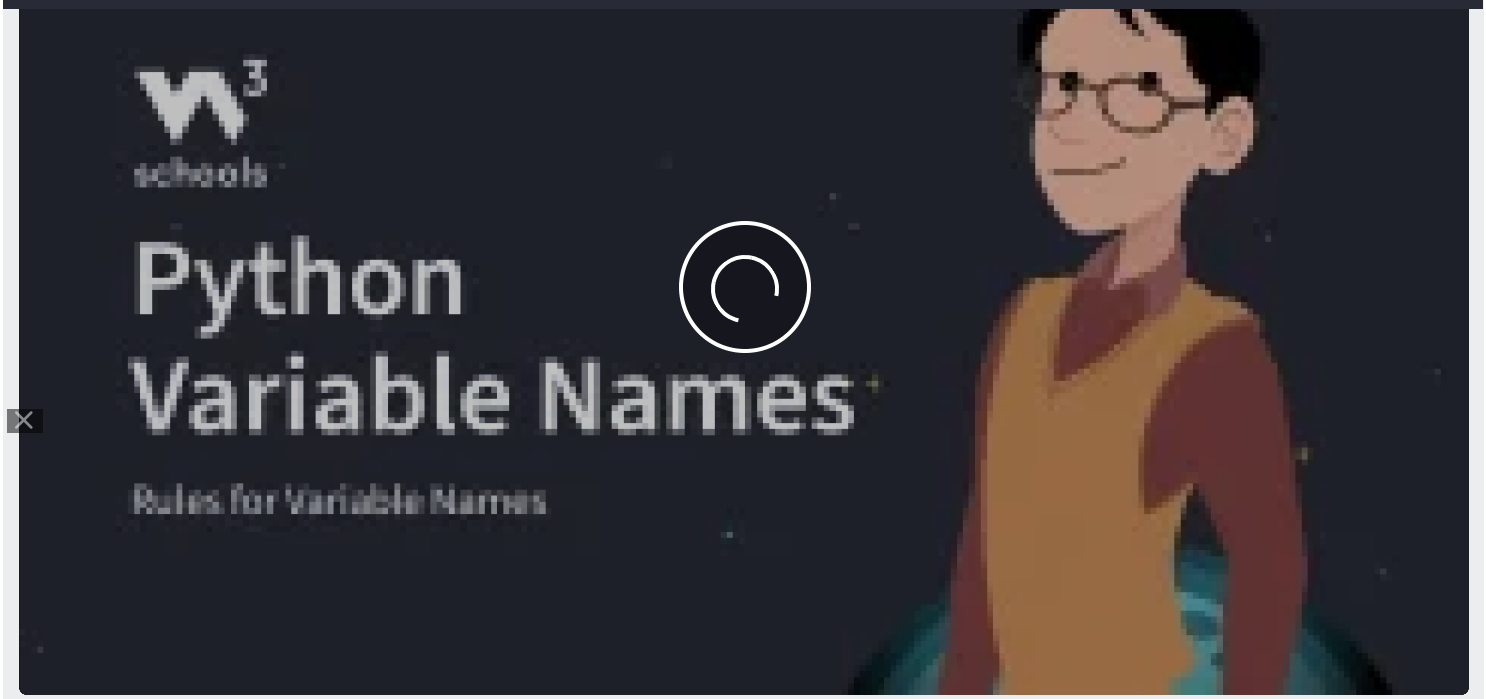
‹ Previous Sign in to track progress Next ›



REMOVE ADS

Tutorials Exercises W3Schools app iOS & Android NEW Sign In

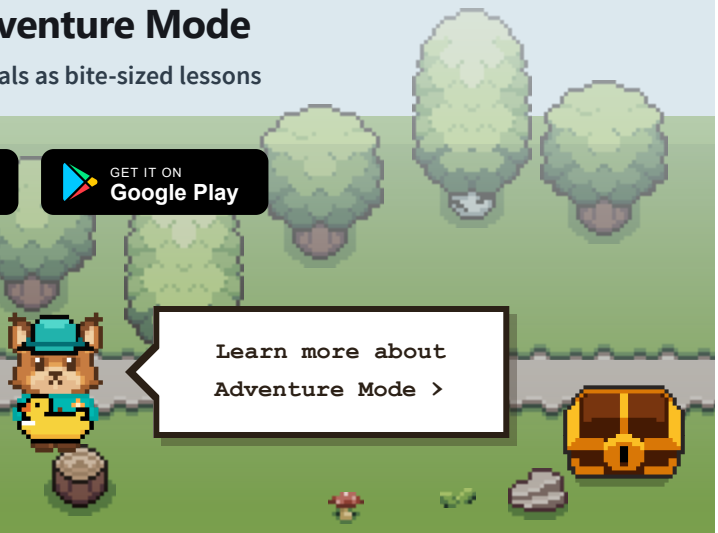
. CSS JAVASCRIPT SQL PYTHON JAVA PHP C C++ C# AWS



Tutorials Exercises
W3Schools app NEW
iOS & Android
Sign In
CSS JAVASCRIPT SQL PYTHON JAVA PHP C C++ C# AWS

FREE ACCOUNT
Track your progress
XP earned 0
Day streak 0
Spaces 0
League —
Sign in to track progress

NEW
Learn in Adventure Mode
Coding fundamentals as bite-sized lessons and challenges.
Download on the App Store
GET IT ON Google Play



PLUS
SPACES
GET CERTIFIED
FOR TEACHERS
PRACTICE
CONTACT US

Top Tutorials
HTML Tutorial
CSS Tutorial
JavaScript Tutorial
How To Tutorial
SQL Tutorial
Python Tutorial
W3.CSS Tutorial
Bootstrap Tutorial
PHP Tutorial
Java Tutorial

Tutorials Exercises W3Schools app iOS & Android NEW Sign In

CSS JAVASCRIPT SQL PYTHON JAVA PHP C C++ C# AWS

CSS Reference
JavaScript Reference
SQL Reference
Python Reference
W3.CSS Reference
Bootstrap Reference
PHP Reference
HTML Colors
Java Reference
AngularJS Reference
jQuery Reference

Top Examples

HTML Examples
CSS Examples
JavaScript Examples
How To Examples
SQL Examples
Python Examples
W3.CSS Examples
Bootstrap Examples
PHP Examples
Java Examples
XML Examples
jQuery Examples

Get Certified

HTML Certificate
CSS Certificate
JavaScript Certificate
Front End Certificate
SQL Certificate
Python Certificate
PHP Certificate
jQuery Certificate
Java Certificate
C++ Certificate
C# Certificate
XML Certificate

FORUM ABOUT CAMPUS

W3Schools is optimized for learning and training. Examples might be simplified to improve reading and learning.
Tutorials, references, and examples are constantly reviewed to avoid errors, but we cannot warrant full correctness
of all content. While using W3Schools, you agree to have read and accepted our terms of use, cookies and privacy policy.

Copyright 1999-2026 by Refsnes Data. All Rights Reserved. W3Schools is Powered by W3.CSS.